Análise da Qualidade do Ar e Clima de Manaus através de Estatísticas de Ordem

Relatório Técnico · ICC002 – Algoritmos e Estruturas de Dados · 2026/2 · Universidade Federal do Amazonas

Alunos: Eduardo Guimarães da Silva Batalha · Matrícula: 22651820
Fernando Naum Santos Batista · Matrícula: 22652386
Data: 22/10/2026

1. Introdução

Este relatório descreve a implementação, em linguagem C, de três algoritmos para encontrar o k-ésimo menor elemento de um conjunto de medições ambientais de Manaus (temperatura, umidade e PM2.5), sem uso de funções prontas de ordenação ou seleção da biblioteca padrão. As três abordagens são distintas:

- **Abordagem A:** ordenação completa pelo método da bolha, seguida do acesso direto à posição k.
- **Abordagem B:** seleção por particionamento (divisão e conquista), com pivô central.
- **Abordagem C:** contagem comparativa sequencial, que conta quantos elementos são menores ou iguais a cada candidato.

As três consultas analisadas são a mediana do PM2.5 (k = N/2), o percentil 90 e os 10 menores valores de umidade. O desempenho é comparado pelo número de comparações para N = 1.000, 5.000 e 10.000 registros.

2. Descrição da implementação

2.1 Entrada e estruturas de dados

Ao iniciar, o programa exibe um menu para escolher o arquivo a ser lido: 1 – 1.000 registros, 2 – 5.000 ou 3 – 10.000. Os arquivos `dados_clima_manaus_1e3.csv`, `dados_clima_manaus_5e3.csv` e `dados_clima_manaus_1e4.csv` são abertos pelo nome com `fopen` e precisam estar na mesma pasta do executável (compilação e execução: `gcc trabalho_clima.c -o trabalho_clima` e `./trabalho_clima`). Como o arquivo é aberto por nome, o teclado fica livre para o segundo menu, no qual se escolhe a abordagem (1 – A, 2 – B, 3 – C, 0 – sair).

A primeira linha (cabeçalho) é descartada e cada registro é lido para uma `struct Registro` com os campos ID_Sensor (int), Data (string), Temperatura_C (float), Umidade (int) e PM2.5 (float). Apenas as colunas usadas nas consultas são guardadas, em dois vetores originais: `pm25[]` e `umid[]` (umidade convertida para float). Os vetores têm capacidade para 10.500 elementos, acima do máximo de 10.000 registros dos arquivos.

Como A e B reorganizam o vetor, elas trabalham sempre sobre uma cópia (`copiar()`), preservando os dados originais. A Abordagem C não altera o vetor e usa o original diretamente.

2.2 Escolha de k

- **Mediana:** k = N/2, conforme a definição adotada no enunciado, considerando k como posição iniciada em 1. Como a divisão em C é inteira, para N ímpar essa regra não corresponde necessariamente à posição central convencional; além disso, N = 1 produziria k = 0. Os testes deste relatório usam N = 1.000, 5.000 e 10.000, para os quais k é válido.
- **Percentil 90:** k = ceil(0,9·N), calculado em inteiros como (9N + 9)/10. Para N = 1.000, k = 900.
- **Dias mais secos:** os M = 10 menores valores de umidade.

2.3 Abordagem A: bolha

O bubble sort percorre o vetor trocando vizinhos fora de ordem. Cada passada leva o maior elemento restante para o fim, então o limite interno diminui a cada volta (`n-1-i`). Há uma otimização: se uma passada completa não faz nenhuma troca, o vetor já está ordenado e o algoritmo para. Depois da ordenação,

mediana e P90 são lidos diretamente em `copia[k-1]`. Uma única ordenação do vetor de PM2.5 responde às duas consultas, por isso seu custo é contado uma só vez.

Os dias mais secos usam outro vetor (umidade). Por isso a Abordagem A faz uma **segunda ordenação**, sobre uma cópia de `umid[]`, e as M primeiras posições são as menores umidades. O custo dessa segunda ordenação é contado separadamente e é o que aparece na tabela da seção 3.3.

2.4 Abordagem B: seleção por particionamento

A cada iteração:

1. o elemento do meio do trecho [inicio, fim] é escolhido como pivô e levado para o fim;
2. o trecho é particionado, deixando à esquerda os elementos menores que o pivô;
3. o pivô é colocado na sua posição final i;
4. se i é a posição procurada (k-1), a busca termina; senão, continua só no lado que contém a posição, descartando o outro.

A escolha do elemento central é feita pela posição no vetor, não pelo valor do elemento. Portanto, ela não garante que o pivô divida os valores em duas partes iguais nem que o trecho restante caia sempre pela metade.

Diferentemente da ordenação, o algoritmo nunca ordena o vetor inteiro: só refina o trecho onde está a resposta. Como a partição reorganiza o vetor, cada consulta (mediana, P90) usa uma cópia nova e seu custo é somado.

Para os M dias mais secos, uma única seleção do M-ésimo menor já deixa os M menores nas M primeiras posições. Elas são ordenadas só para exibição, e esse custo não entra na contagem.

2.5 Abordagem C: contagem comparativa sequencial

Para cada elemento x, na ordem do vetor, o algoritmo varre todo o vetor contando quantos elementos são menores que x (`menores`) e quantos são iguais a x (`iguais`, incluindo ele mesmo). Se x fosse ordenado, ocuparia as posições `menores+1` até `menores+iguais`. Logo, x é a resposta quando:

```
menores < k  e  k <= menores + iguais
```

O algoritmo para no primeiro elemento que satisfaz a condição. Não altera o vetor, mas refaz a varredura completa a cada consulta. Para os dias mais secos, executa uma busca para cada k = 1..M.

2.6 Como as comparações são contadas

- **A:** cada teste `vetor[j] > vetor[j+1]`. Cada ordenação (PM2.5 e umidade) é contada à parte.
- **B:** cada teste `vetor[j] < pivo` dentro da partição.
- **C:** cada teste `vetor[j] < x` e, quando este falha, também o teste `vetor[j] == x`. Assim, C custa de 1 a 2 comparações por par examinado.

Por isso, a contagem de C pode chegar a duas comparações por elemento examinado, enquanto uma contagem que considerasse apenas um teste por par registraria menos operações.

3. Resultados experimentais

3.1 Corretude

Para cada tamanho de arquivo, foram executadas as opções 1 (A), 2 (B) e 3 (C) do menu e comparadas as saídas. As três abordagens devolveram exatamente os mesmos valores:

N	Mediana (PM2.5)	P90 (PM2.5)	10 dias mais secos (umidade, %)
1.000	59,0	187,0	15 15 15 15 15 15 15 15 15 15
5.000	59,0	191,0	15 15 15 15 15 15 15 15 15 15
10.000	60,0	193,0	15 15 15 15 15 15 15 15 15 15

O valor 15 repetido dez vezes indica que a umidade mínima registrada (15%) aparece em pelo menos dez registros. Há muitos empates nesse extremo, e as três abordagens os trataram de forma consistente.

3.2 Número de comparações: mediana + P90

Para B e C, o valor é a soma das duas buscas. Para A, é uma única ordenação do vetor de PM2.5.

N	A (bolha)	B (partição)	C (contagem)
1.000	498.372	7.037	805.926
5.000	12.488.184	25.519	2.320.764
10.000	49.981.305	49.578	2.945.470

3.3 Número de comparações: 10 dias mais secos

Para A, o valor é o custo de uma segunda ordenação, feita sobre o vetor de umidade.

N	A (bolha)	B (partição)	C (contagem)
1.000	499.094	1.645	270.790
5.000	12.485.254	10.196	4.071.890
10.000	49.934.274	13.093	15.568.030

3.4 Análise dos resultados

Abordagem A. Segue quase exatamente o valor teórico N(N-1)/2: 499.500 para N = 1.000, 12.497.500 para N = 5.000 e 49.995.000 para N = 10.000. Na mediana + P90, o valor medido fica de cerca de mil a cerca de 14 mil comparações abaixo; nos dias mais secos, de cerca de 400 a cerca de 61 mil abaixo (sempre menos de 0,3% do valor teórico), graças à parada antecipada. Ao multiplicar N por 10 (de 1.000 para 10.000), as comparações crescem cerca de 100 vezes (498.372 para 49.981.305), o que é o comportamento quadrático clássico.

Abordagem B. É de longe a menor: com N = 10.000, são 49.578 comparações para duas consultas, cerca de 5N, ou seja, 2 a 3 varreduras do vetor por consulta. Ao multiplicar N por 10, o custo cresceu pouco mais de 7 vezes, compatível com crescimento linear. Em relação à bolha, B fez cerca de 1.000 vezes menos comparações com N = 10.000 (49.981.305 contra 49.578).

Abordagem C. O custo varia conforme a posição do primeiro candidato que satisfaz a condição de busca. Por isso, as comparações de mediana e P90 não crescem de forma regular nos testes. Na consulta dos dias mais secos, o crescimento observado foi de cerca de 57 vezes ao passar de 1.000 para 10.000 registros (270.790 para 15.568.030). Esse resultado é compatível com um comportamento próximo do quadrático neste conjunto de testes, mas não comprova sozinho a complexidade assintótica.

4. Análise teórica: melhor e pior caso

São consideradas N medições e a busca pelo k-ésimo menor.

Algoritmo	Melhor caso	Pior caso	Caso médio / observação
A – Bolha (com parada antecipada)	N − 1 comparações: vetor já ordenado, uma passada sem trocas. $\Theta(N)$	$N(N-1)/2$ comparações: vetor em ordem decrescente. $\Theta(N^2)$	$\Theta(N^2)$, cerca de $N^2/2$ para dados aleatórios. É o que se observa nos testes.
B – Particionamento (pivô central)	N − 1 comparações: o primeiro pivô já cai na posição k (ex.: dados ordenados e k na metade). $\Theta(N)$	Cerca de $N^2/2$ comparações: o pivô cai sempre num extremo e o trecho encolhe só 1 por etapa. $\Theta(N^2)$	$\Theta(N)$: cerca de 3N por consulta (de 2,5N a 3,5N nos testes). O pior caso com pivô central é raro.
C – Contagem comparativa	Entre N e 2N comparações: a resposta é o primeiro elemento examinado. $\Theta(N)$	Cerca de $(3N^2 + N)/2 \approx 1{,}5N^2$ (valores distintos), podendo chegar a $2N^2$: a resposta é o último elemento examinado. $\Theta(N^2)$	$\Theta(N^2)$ na prática, pois depende da posição do valor no arquivo.

Observações:

- **Bolha:** a diferença entre o melhor e o pior caso é enorme. O pior caso é o de dados em ordem decrescente, e os dados reais (aleatórios) ficam muito perto dele.

- **Particionamento:** o pior caso com pivô central ocorre em permutações específicas. Também ocorre se todos os valores forem iguais: como a partição usa <, nenhum elemento é menor que o pivô, que fica sempre no início, e o trecho encolhe de um em um. Nos dados deste trabalho, que têm repetições mas não são constantes, isso não foi problema.
- **Contagem:** para valores distintos, o custo de varrer todo o vetor é 2N − menores por elemento, e a soma de menores sobre todos os elementos vale N(N−1)/2, o que dá o valor 1,5N² citado.

5. Custo da Abordagem C frente ao particionamento (B)

A cada etapa, B descarta o lado que não contém a posição procurada. Se as partições forem equilibradas, o algoritmo reduz o trecho rapidamente e pode ter custo linear. No entanto, o código escolhe o pivô pela posição central, sem garantir que ele divida os valores de forma equilibrada. Em partições ruins, o trecho pode diminuir apenas um elemento por etapa, levando a custo quadrático.

A Abordagem C, ao contrário, não aproveita nada do que já viu. Para decidir se um candidato é a resposta, ela precisa comparar esse candidato com todos os outros N elementos. Como o candidato certo pode ser qualquer um, o algoritmo tem N candidatos × N comparações, ou seja, laços aninhados e custo quadrático. Além disso, cada comparação é contada duas vezes quando o primeiro teste falha, e a busca é refeita do zero para cada nova consulta (mediana, P90, e cada um dos 10 dias mais secos).

O impacto cresce muito rápido com o volume de dados. Para um pior caso de cerca de 1,5N² comparações por consulta:

N	B (≈ 3N por consulta)	C no pior caso (≈ 1,5N² por consulta)	Razão C/B
10.000	≈ 30 mil	≈ 150 milhões	≈ 5.000×
1.000.000	≈ 3 milhões	≈ 1,5 trilhão	≈ 500.000×

Usando como referência um computador que execute cerca de 10^9 comparações por segundo, B levaria milissegundos para um milhão de registros, enquanto C poderia levar cerca de 25 minutos por consulta no pior caso estimado. Para bases muito grandes, isso torna C pouco prática. No pior caso, C também pode realizar mais comparações que a bolha: aproximadamente 1,5N² a 2N², contra cerca de N²/2.

6. Respostas às questões de acompanhamento

Questão 1: rastreando a Abordagem C

Lista: [45, 90, 30, 90, 60], N = 5, k = 3. O vetor ordenado seria [30, 45, 60, 90, 90], logo a resposta esperada é 60. O algoritmo examina cada elemento x na ordem em que aparece, varrendo os 5 valores e contando *menores* e *iguais*:

i	x	menores	iguais	Posições que x ocuparia	Comparações	k = 3 satisfeito?
0	45	1 (o 30)	1	2	9	Não: 3 ≤ 1+1 é falso
1	90	3 (45, 30, 60)	2	4 a 5	7	Não: 3 < 3 é falso
2	30	0	1	1	10	Não: 3 ≤ 0+1 é falso
3	90	3	2	4 a 5	7	Não: 3 < 3 é falso
4	60	2 (45, 30)	1	3	8	Sim: 2 < 3 ≤ 3, retorna 60

Total: 9 + 7 + 10 + 7 + 8 = 41 comparações, na contagem do código implementado (cada teste < mais o teste == quando o < falha). Se fosse contado apenas um teste por par (N × N, até achar a resposta), seriam 5 elementos × 5 = 25 comparações. A diferença vem apenas do critério de contagem. Cada elemento custa 2N − menores = 10 − menores comparações, o que confere com a tabela (9, 7, 10, 7, 8).

Questão 2: o problema dos empates

O que acontece se contarmos só os estritamente menores (<)? Sem tratar empates, a condição de parada seria `menores == k − 1`. Funciona para elementos únicos, mas quebra com repetições. Na lista da Questão 1, os dois 90 têm menores = 3; assim, para k = 4 o primeiro 90 serve, mas para k = 5 seria

necessário um elemento com menores = 4, que não existe. Nenhum elemento satisfaz a condição e o algoritmo não encontra resposta. Se o contador fosse de menores ou iguais (≤) e a condição contagem == k, ocorre o problema simétrico: os 90 teriam contagem 5, e k = 4 nunca seria satisfeito.

Como a lógica foi ajustada. Em vez de um único contador, o código usa dois: `menores` (estritamente menores) e `iguais` (iguais, incluindo o próprio elemento). Um valor repetido não ocupa uma única posição no vetor ordenado, mas um intervalo de posições, de `menores+1` até `menores+iguais`. O elemento é a resposta quando k cai dentro desse intervalo: `menores < k <= menores + iguais`. Como esses intervalos cobrem todas as posições de 1 a N sem lacunas, sempre existe um valor que satisfaz a condição para qualquer 1 ≤ k ≤ N. O laço externo é limitado por `i < n`, então o algoritmo nunca fica preso, e o `return -1` final é apenas uma segurança para k inválido. Na prática, isso apareceu nos testes: a consulta dos dias mais secos retornou o valor 15 para todos os k de 1 a 10, porque 15 ocupa todas essas posições.

Questão 3: o impacto da escolha do pivô (Abordagem B)

Dados já ordenados e pivô sempre no primeiro elemento. Com o vetor em ordem crescente, o primeiro elemento do trecho é o menor de todos. A partição percorre o trecho inteiro (N − 1 comparações), mas nenhum elemento é menor que o pivô, que fica onde estava. O espaço de busca diminui apenas 1 elemento por etapa, em vez de cair pela metade. O total vira (N−1) + (N−2) + (N−3) + ..., ou seja, um custo quadrático, da mesma ordem da bolha. Esse exemplo ilustra uma estratégia de pivô fixo no primeiro elemento, não a implementação entregue. No código deste trabalho, o pivô é sempre o elemento central do trecho; em um vetor ordenado, se a posição central é procurada, ele pode encontrar a mediana na primeira partição.

Por que o pivô aleatório pode ajudar. A escolha de um pivô aleatório é uma possível melhoria teórica, mas não está implementada neste programa: o código seleciona sempre o elemento do meio do trecho, pela posição. Com pivô aleatório, a escolha deixa de depender diretamente da ordem original dos dados e, sob as hipóteses usuais da análise aleatorizada, o custo esperado da seleção é linear. Isso não elimina o pior caso quadrático; torna menos provável que uma entrada específica provoque repetidamente partições muito desequilibradas. Essa explicação é uma comparação teórica, não uma descrição do código entregue.

Questão 4: observando o gargalo na prática

Ao passar de N = 1.000 para N = 10.000, a abordagem cujo custo cresceu de modo mais previsível e mais intenso foi a A (bolha): de 498.372 para 49.981.305 comparações, ou seja, 100 vezes para 10 vezes mais dados. A Abordagem C aparece em seguida e, na consulta dos dias mais secos, cresceu cerca de 57 vezes (de 270.790 para 15.568.030). Já B ficou na casa de dezenas de milhares de comparações. Nos testes realizados, portanto, A e C foram as que mais sentiram o aumento, e B quase não sentiu.

A Abordagem C merece atenção especial por um motivo teórico. Embora, na maior parte dos dados de teste (exceto em mediana + P90 com N = 1.000, em que fez 805.926 contra 498.372), ela tenha feito menos comparações que a bolha (por parar assim que acha a resposta), seu pior caso é pior que o da bolha: cerca de $1{,}5N^2$ a $2N^2$ comparações, contra $N^2/2$ (seção 4). A estrutura explica o problema: um laço sobre os candidatos (N) contendo outro laço de comparação com todos os elementos (N) produz N × N operações. A bolha também tem dois laços aninhados, com custo de mesma ordem.

Se o Instituto Ambiental decidir processar milhões de medições, N^2 passa a ser da ordem de 10^{12} operações. Quando o volume é multiplicado por 10, o custo é multiplicado por 100; um cálculo que levava segundos passa a levar minutos ou horas, sem falar que, com várias consultas, o custo se multiplica. Já o particionamento (B), de custo linear, apenas acompanha o crescimento dos dados.

Questão 5: reutilização vs. velocidade única

Com a Abordagem A: o programa não precisa ordenar de novo. Depois da ordenação completa, o vetor fica ordenado e qualquer estatística de ordem é uma simples leitura da posição desejada, com custo constante. No código, é o que acontece: uma única ordenação serve para a mediana e para o P90 (`copia[k_mediana-1]` e `copia[k_p90-1]`), e as comparações são contadas uma única vez.

Com a Abordagem B: o comportamento é diferente. A seleção não ordena o vetor; ela apenas reorganiza parcialmente o trecho onde está a resposta. Depois de achar a mediana, o vetor fica só parcialmente

arrumado e não permite ler o P90 diretamente. Na implementação adotada, cada consulta recebe uma cópia nova e faz uma nova busca do zero, e os custos somam (7.037 comparações para N = 1.000, soma das duas buscas). Seria possível reaproveitar o trabalho já feito: como o P90 está à direita da mediana, bastaria continuar a busca só no lado direito. Essa melhoria não foi implementada.

Diferença prática. A ordenação tem custo alto uma vez e depois responde a qualquer número de consultas quase de graça. A seleção tem custo baixo a cada consulta. Com a bolha, a ordenação custa cerca de $N^2/2$ comparações (49.981.305 com N = 10.000), enquanto cada seleção custa em torno de 2,5N nos testes (cerca de 25 mil). Mesmo assim, o particionamento vence com folga: para a ordenação compensar, seriam necessárias mais de 2.000 consultas seguidas. Com poucas consultas (como as duas do enunciado), a seleção é muito mais vantajosa. A ordenação só passa a valer a pena se o número de consultas for grande e o algoritmo de ordenação for eficiente (N log N, e não a bolha).

7. Conclusão

- As três abordagens produziram resultados idênticos para mediana, P90 e dias mais secos nos três tamanhos de entrada, o que valida as implementações.
- A bolha (A) mostrou crescimento quadrático exato nos testes (100× ao multiplicar N por 10), embora tenha a vantagem de ordenar tudo e responder a novas consultas sem custo adicional.
- O particionamento (B) é a melhor escolha para poucas consultas sobre grandes volumes: custo linear, cerca de 1.000 vezes menos comparações que a bolha em N = 10.000.
- A contagem comparativa (C) tem pior caso quadrático (cerca de $1{,}5N^2$ a $2N^2$), pior até que o da bolha. Os valores baixos observados para ela em parte dos testes se devem à parada antecipada e à posição dos valores no arquivo, e não a uma eficiência real do método. Ela só é aceitável para N pequeno.